Online Academic Catalog

Table of Content:

1. **Introduction**
   1.1. Project Overview
   1.2. Objectives
2. **Project Arhitecture**
3. **Database**
   3.1. Database Overview
   3.2. Tables and Relationship
4. **Implementation**
   4.1. Models and Design Decisions
   4.2. Features
5. **Instructions for running**
6. **Challenge and solutions**
7. **Bibliography**

1.Introduction

1.1. Project Overview

The Online Academic Catalog is a comprehensive Java-based desktop solution designed to modernize and streamline the academic recording process within educational institutions. Developed using the Java Swing framework for the graphical user interface and PostgreSQL for robust data persistence, the application serves as a centralized digital hub for academic data management. The system is built to replace traditional, error-prone manual grading methods with a synchronized environment where data integrity is paramount. It addresses the needs of three distinct user categories: Administrators, Teachers, Students. This project is significantly more resilient to manual entry errors typically found in physical catalogs, as it allows for easier editing and management of data. It offers a more practical user experience because the application automates most of the complex logic and background tasks on behalf of the user.

1.2. Objectives

The primary goal of this project is to transition from a rigid, manual academic recording process to a flexible and automated digital environment. The specific objectives are as follows:

- **Error Reducing and Ease of Correction**: Unlike traditional paper-based catalogs, this system is designed to be much more "error-friendly," allowing authorized users to edit and update records effortlessly
- **Operational Automation**: To minimize manual workload by automating complex tasks such as weighted average calculations, credit tracking, and institutional email generation.
- **Role-Based Data Management**:
    1. **For Administrators**: To provide a "mission control" interface for rapid user enrollment, group assignments, and system-wide data management.
    2. **For Teachers**: To offer a practical tool for real-time grading and attendance tracking, ensuring that teachers can focus on academic delivery rather than administrative paperwork.

3. **For Students**: To grant 24/7 access to their personal academic profile, enabling them to monitor their progress, grades, and promotion status independently

- **Data Integrity and Validation**: To implement strict logic in the application layer that prevents common mistakes, such as assigning grades to subjects a teacher does not coordinate or double-booking records for the same student.
- **Improved User Experience**: To create an intuitive interface where the application "does almost everything for you," making it significantly more practical and efficient to use than previous methods.

2.Project Arhitecture

The application uses a Layered Arhitecture to separate the user interface from the data storage. This structure makes the system "practical" and "automated" by giving each part a clear responsibility.

a. The Layers:

- **UI Layer (Java Swing)**: This is the "face" of the application where the user interacts with buttons and tables (e.g. “LoginFrame”, “GroupViewFrame”). It focuses on displaying results rather than performing calculations.
- **Business Logic Layer (Models)**: This is the "brain" that does almost everything for the user. It automatically calculates averages, validates grade entries, and generates student emails.
- **Data Access Layer (JDBC)**: This part connects the Java code to the database through the “DBConnection” class. It handles sending and saving data to PostgreSQL.
- **Database Layer (PostgreSQL)**: The secure location where all information for students, teachers, and grades, which is permanently stored.

3. Database

3.1. Database Overview

The system uses a relational PostgreSQL database named “CatalogOnline”. It is structured to maintain data integrity; for example, foreign key constraints prevent the deletion of a student record if associated grades still exist in the catalog.

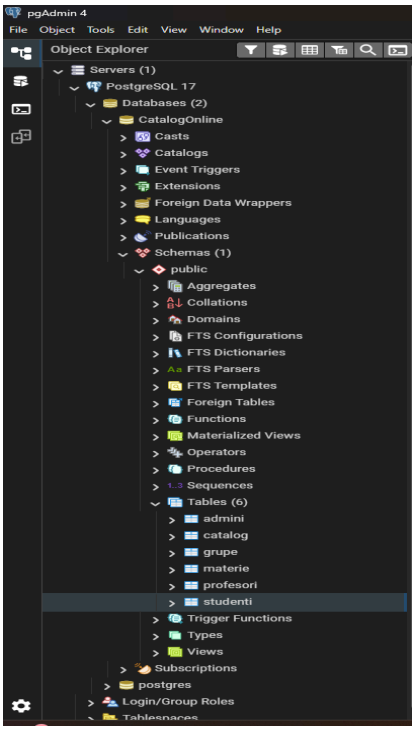
	id_materie [PK] integer	id_profesor integer	nume_materie character varying (100)	limba integer	an integer	credite integer
1	1	10	Proiectare Logica	1	1	6
2	2	11	Matematici Speciale	1	1	6
3	3	12	Programarea Calculatoarelor	1	1	7
4	4	13	Electrotehnica	1	1	6
5	5	14	Calcul Diferential	1	1	5
6	6	15	Logic Design	2	1	6
7	7	11	Special Matematics	2	1	6
8	8	16	Computer Programming	2	1	7
9	9	13	Electrotechnics	2	1	6
10	10	17	Differential Calculus	2	1	5
11	11	20	Algoritmi Fundamentali	1	2	7
12	12	21	Programare Orientata Obiect	1	2	6
13	13	22	Metode Numerice	1	2	5

tabelul de “materi”

3.2. Tables and Relationship

Based on the pgAdmin schema and the source code, there are 6 main tables:

1. **admini**: Stores administrator credentials.
2. **profesori**: Teacher data and teaching language specializations.
3. **studenti**: Stores student details and their group affiliation (id_grupa).
4. **materie**: Lists courses, credit points and the assigned teacher.
5. **grupe**: Defines study groups organized by year and language.
6. **catalog**: The central junction table linkin students, subjects, and teachers to record marks (“Nota”) and absences (“Absenta”).



4. Implementation

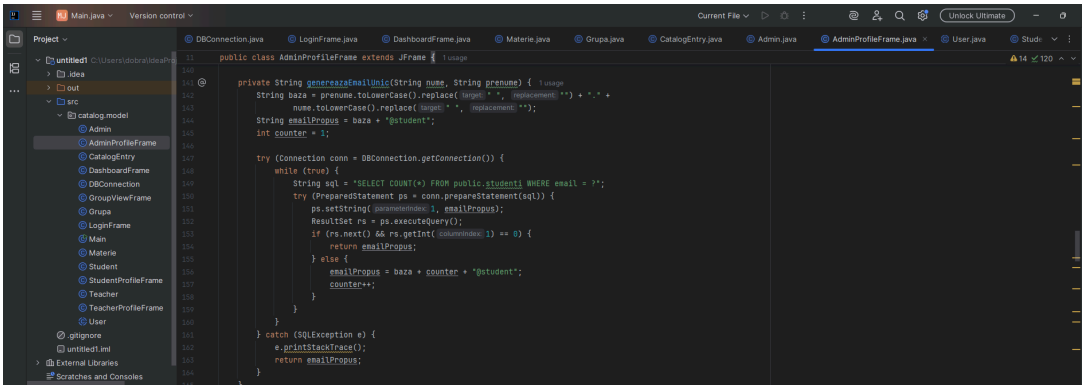
4.1. Models and Design Decisions

The system follows a Domain-Driven Design approach, where each class in the model layer directly corresponds to a table in the PostgreSQL database.

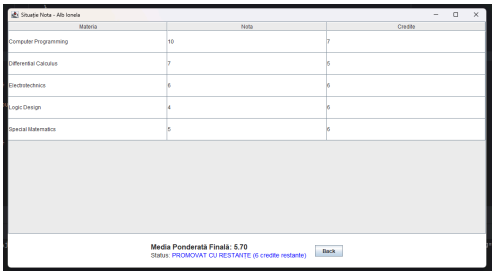
- **Inheritance(User Abstract Class):**
    1. **Decision**: We implemented an abstract “User” class to avoid code duplication.
    2. **Benefit**: Attributes like: “id”, ”nume”, ”email” and ”parola” are inherited by: “Admin”, “Teacher”, and “Student”, ensuring a uniform authentication process in the “Main.findUser()” method.
- **Admin Model:**
    1. **Role**: Manages the ecosystem.
    2. **Automation**: It encapsulates the ”genereazaEmailUnic()” logic, which is a key design decision to automate the creation of institutional identifiers without risking database collisions.
- **Teacher & Student Models**:
    1. **Teacher**: Contains logic for academic tenure, ensuring teachers only interact with the groups and subjects assigned to them in the database.
    2. **Student**: Includes the “id_grupa” attribute, which acts as a key for the automated calculation of weighted averages based on the specific curriculum of that group.
- **CatalogEntry (Data Transfer Object)**:
    1. **Decision**: Instead of passing raw strings or numbers between the UI and the Database, we used the “CatalogEntry” object.
    2. **Benefit**: This simplifies the “Insert” and “Update” operations in the “catalog” table, making the code much cleaner and less prone to errors.

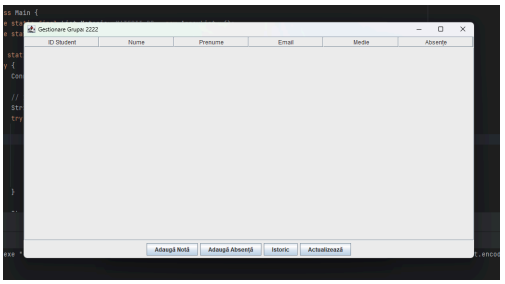
4.2. Features:

1. **Automated Email Generation**: Dynamically creates unique institutional emails during student registration (e.g. firstname.lastname@student)



2. **Weighted Average Engine**: Automatically calculates student averages based on subject credits and numerical grades.
3. **Promotion Status Tracking**: Determines "Pass/Fail" status based on total credits remaining.

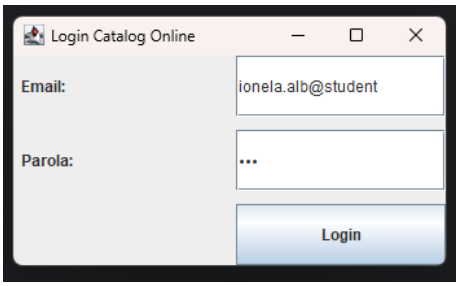




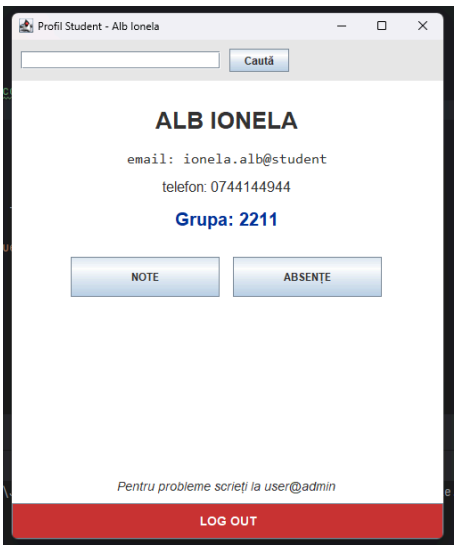
4. **Attendance Monitoring**: A specialized interface for teachers to record and students to view absences per subject.
5. **Real-time Filtering**: Dynamic search functionality allows users to filter through records instantly as they type.
6. **Transactional Data Safety**: Uses SQL transactions to ensure that student deletions are only finalized once all associated catalog entries are removed.

5. Instructions for running

1. **DB Setup**: Create the “CatalogOnline” database in PostgreSQL.
2. **Configuration**: Enter the local database credentials (URL, user, password) into the “DBConnection.java” class.
3. **Execution**: Run the “Main” class to access the initial “LoginFrame”.



partea de login



profil student

6.Challenge and solutions

- **Challenge**: Deleting students with existing academic history.
- **Solution**: Implemented manual SQL transactions that clear dependent records in the “catalog” table before removing the student profile
- **Challenge**: Ensuring teachers only grade their assigned courses.
- **Solution**: UI-level validation that matches the logged-in teacher's ID with the course's tenure in the database

7.Bibliography

- Materials and notes from courses and labs.
- https://www.youtube.com/playlist?list=PLZPZq0r_RZOOj_NOZYq_R2PECIMglLemc
- https://www.w3schools.com/postgresql/index.php
- https://www.w3schools.com/java/
- https://docs.oracle.com/javase/tutorial/